

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Experiments

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 1 – Experiments
Weightage:	50% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	Y Greeshma Reddy	Reg. No.:	192424146
Section / Batch:	B Tech-AI&DS	Date:	06-08-2026

Code of Conduct

I Y Greeshma Reddy (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Y Greeshma Reddy

Instructions

- Perform all experiments using Google Colab.
- Create a separate notebook (.ipynb) for the assessment.
- Ensure all code is executable without errors.
- Include appropriate comments explaining the logic used.
- Complete all three experiments in a single Google Colab notebook.
- Execute all cells and display the outputs for the given test cases.
- Save the notebook with the naming convention: RegNo_Name_CO3-AT1.ipynb
- Upload the notebook to your GitHub repository.
- Ensure the repository is public or accessible to the instructor.
- Submit the following:
 - GitHub Repository Link in GCR
 - Google Colab Share Link in GCR
 - Screenshots of uploaded coding and output files as printout.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
Context-Free Grammars for English Syntax, Context-Free Rules and Trees, Sentence-Level Constructions, Parsing, Top-Down Parsing, Earley Parsing	Design a Python program to validate sentences using CFG production rules and construct parse tree representations for valid sentences.	Q1
Agreement, Feature Structures, Sub Categorization	Design a Python program to verify subject-verb agreement and validate grammatical constraints using feature structures and subcategorization rules.	Q2
Probabilistic Context-Free Grammars (PCFGs)	Design a Python program to parse sentences and compute sentence probabilities using probabilistic grammar production rules.	Q3

Annexure A – Experiments

Experiment 1: Context-Free Grammar Rule Validation and Parse Tree Construction

Experiment Description

Design a Python program to validate whether a sentence follows the given Context-Free Grammar (CFG) rules and generate its parse tree representation.

Grammar:

```
S → NP VP
NP → Det N
VP → V NP

Det → the | a
N → student | teacher | book
V → reads | likes
```

Task

1. Accept a sentence as input.
2. Verify whether it conforms to the CFG.
3. Construct and return the parse tree.
4. Return "Invalid Sentence" if parsing fails.

Function Prototype

```
def parse_cfg(sentence):
    pass
```

Test Cases

Input	Expected Output
the student reads a book	Valid Parse Tree
a teacher likes the book	Valid Parse Tree
student reads book	Invalid Sentence
the book likes a teacher	Valid Parse Tree
reads the student book	Invalid Sentence

Experiment 2: Agreement and Feature Structure Checking

Experiment Description

Design a Python program that verifies subject–verb agreement using feature structures containing Number and Person attributes.

Feature Rules

```
he → singular, third person
she → singular, third person
it → singular, third person
they → plural

runs → singular verb
writes → singular verb

run → plural verb
write → plural verb
```

Task

1. Accept Subject and Verb.

2. Compare feature structures.
3. Return True if agreement holds.
4. Return False otherwise.

Function Prototype

```
def check_agreement(subject, verb):
    pass
```

Test Cases

Subject Verb Expected

he	runs	True
he	run	False
they	run	True
they	writes	False
she	writes	True

Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

Experiment Description

Implement a Python program to compute the probability of a sentence using the given PCFG.

Grammar

$S \rightarrow NP VP$	[1.0]
$NP \rightarrow \text{John}$	[0.6]
$NP \rightarrow \text{Mary}$	[0.4]
$VP \rightarrow \text{runs}$	[0.5]
$VP \rightarrow \text{walks}$	[0.5]

Probability Formula

$$\begin{aligned}
 &P(\text{Sentence}) \\
 &= \\
 &P(S \rightarrow NP VP) \\
 &\times P(NP) \\
 &\times P(VP)
 \end{aligned}$$

Task

1. Accept a two-word sentence.
2. Parse using the PCFG.
3. Compute sentence probability.
4. Return 0 if sentence cannot be generated.

Function Prototype

```
def sentence_probability(sentence):
    pass
```

Test Cases

Input Sentence Expected Probability

John runs	0.30
John walks	0.30
Mary runs	0.20
Mary walks	0.20
Peter runs	0.00

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Experiment 1: Context-Free Grammar (CFG) Rule Validation and Parse Tree Construction

1. Understanding of Concepts

- Understand the concept of Context-Free Grammar (CFG) and its production rules.
- Learn the difference between terminal and non-terminal symbols.
- Understand how grammar rules are used to generate valid sentences.
- Understand the process of sentence parsing using CFG.
- Learn the importance of parse trees in representing sentence structure.

2. Experimental Design

- Design a program to accept a sentence as user input.
- Validate the sentence using the given CFG production rules.
- Check each word according to the grammar sequence.
- Generate a parse tree for grammatically correct sentences.
- Display "Invalid Sentence" when the sentence does not satisfy the grammar rules.

3. Use of Tools

- Use Python programming language for implementation.
- Use functions to organize the program.
- Apply string manipulation techniques to process input.
- Use lists and conditional statements for grammar validation.
- Execute the program successfully in Python IDLE, VS Code, or Google Colab.

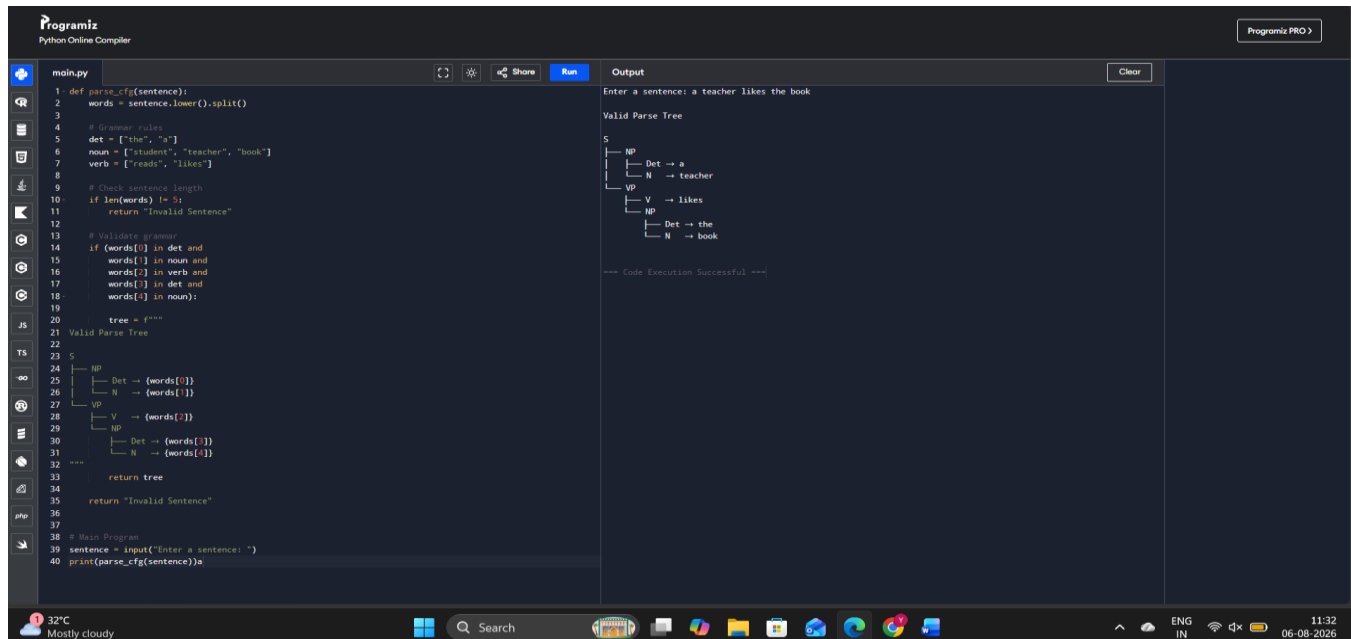
4. Analysis of Results

- Verify whether the input sentence follows the given grammar.
- Compare the generated output with the expected results.
- Confirm that valid sentences produce the correct parse tree.
- Ensure invalid sentences are identified correctly.
- Evaluate the correctness and reliability of the parser using different test cases.

5. Documentation

- Clearly state the objective of the experiment.
- Describe the CFG rules used in the program.
- Explain the algorithm or working procedure.
- Include the complete Python source code.
- Record sample inputs, outputs, and conclusion.

IMPLEMENTATION:



```
1: def parse_cfg(sentence):
2:     words = sentence.lower().split()
3:
4:     # Grammar rules
5:     det = ["the", "a"]
6:     noun = ["student", "teacher", "book"]
7:     verb = ["reads", "likes"]
8:
9:     # Check sentence length
10:    if len(words) != 5:
11:        return "Invalid Sentence"
12:
13:    # Validate grammar
14:    if (words[0] in det and
15:        words[1] in noun and
16:        words[2] in verb and
17:        words[3] in det and
18:        words[4] in noun):
19:
20:        tree = """
21: Valid Parse Tree
22:
23: S
24: |
25: |--- NP
26: |   |--- Det --> {words[0]}
27: |   |--- N --> {words[1]}
28: |--- VP
29: |   |--- V --> {words[2]}
30: |   |--- NP
31: |       |--- Det --> {words[3]}
32: |       |--- N --> {words[4]}
33: """
34:        return tree
35:    else:
36:        return "Invalid Sentence"
37:
38: # Main Program
39: sentence = input("Enter a sentence: ")
40: print(parse_cfg(sentence))
```

Enter a sentence: a teacher likes the book

Valid Parse Tree

```
S
|
|--- NP
|   |--- Det --> a
|   |--- N --> teacher
|--- VP
|   |--- V --> likes
|   |--- NP
|       |--- Det --> the
|       |--- N --> book
```

==== Code Execution Successful ====

Experiment 2: Agreement and Feature Structure Checking

1. Understanding of Concepts

- Understand the concept of subject–verb agreement in English grammar.
- Learn the role of feature structures in Natural Language Processing.
- Understand grammatical features such as number and person.
- Learn how agreement rules are applied between subjects and verbs.
- Recognize correct and incorrect grammatical combinations.

2. Experimental Design

- Design a program to accept a subject and a verb as input.
- Store grammatical features for subjects and verbs.
- Compare the feature structures of both inputs.
- Return **True** when the agreement is correct.
- Return **False** when the agreement rules are violated.

3. Use of Tools

- Use Python dictionaries to store feature information.
- Use functions for modular program design.
- Apply conditional statements for feature comparison.
- Use input and output functions for user interaction.
- Execute the program successfully in any Python environment.

4. Analysis of Results

- Verify whether the subject and verb agree grammatically.

- Test the program using both valid and invalid inputs.
- Compare the obtained results with expected outputs.
- Ensure the agreement checking logic works correctly.
- Evaluate the accuracy of the feature-based approach.

5. Documentation

- Mention the objective of the experiment.
- Explain the feature structure rules used.
- Describe the algorithm followed by the program.
- Include the complete Python implementation.
- Record test cases, outputs, and conclusion.

IMPLEMENTATION:

The screenshot displays the Programiz Python Online Compiler interface. The editor on the left contains a Python script named `main.py` that defines a function `check_agreement(subject, verb)` to verify if a subject and verb agree in number (singular or plural). The script includes dictionaries for subject and verb features and a main program that takes user input for a subject and verb, calls the function, and prints the result.

```

1: def check_agreement(subject, verb):
2:
3:     # Subject Features
4:     subjects = {
5:         "he": "singular",
6:         "she": "singular",
7:         "it": "singular",
8:         "they": "plural"
9:     }
10:
11:     # Verb Features
12:     verbs = {
13:         "runs": "singular",
14:         "writes": "singular",
15:         "run": "plural",
16:         "write": "plural"
17:     }
18:
19:     if subject not in subjects:
20:         return False
21:
22:     if verb not in verbs:
23:         return False
24:
25:     return subjects[subject] == verbs[verb]
26:
27:
28: # Main Program
29: subject = input("Enter Subject: ").lower()
30: verb = input("Enter Verb: ").lower()
31:
32: result = check_agreement(subject, verb)
33:
34: print("Agreement:", result)

```

The output panel on the right shows the execution results for the input "he" and "runs", confirming that the agreement is true. The interface also includes a file explorer on the left, a search bar, and a status bar at the bottom showing system information like temperature and time.

Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

1. Understanding of Concepts

- Understand the concept of Probabilistic Context-Free Grammar (PCFG).
- Learn how probabilities are assigned to grammar rules.
- Understand the probability calculation for sentence generation.
- Learn how statistical parsing differs from normal CFG parsing.
- Recognize sentences that cannot be generated by the grammar.

2. Experimental Design

- Design a program to accept a two-word sentence.

- Parse the sentence using the given PCFG rules.
- Calculate the probability using the specified formula.
- Return the correct probability for valid sentences.
- Return **0** when the sentence cannot be generated.

3. Use of Tools

- Use Python dictionaries to store grammar probabilities.
- Use arithmetic operations to compute sentence probability.
- Use functions for modular implementation.
- Apply string operations to process input sentences.
- Execute the program successfully without external libraries.

4. Analysis of Results

- Verify the calculated probabilities for valid sentences.
- Ensure invalid sentences return zero probability.
- Compare computed values with expected probabilities.
- Validate the correctness of probability calculations.
- Evaluate the efficiency and accuracy of the parser.

5. Documentation

- State the objective of the experiment.
- Explain the PCFG rules and probability formula.
- Describe the algorithm used for parsing.
- Include the complete Python source code.
- Record sample inputs, outputs, and conclusion.

IMPLEMENTATION:

The screenshot displays the Programiz Python Online Compiler interface. The editor on the left contains the following Python code:

```

1 def sentence_probability(sentence):
2     words = sentence.split()
3     # Sentence must contain exactly two words
4     if len(words) != 2:
5         return 0.0
6     # Grammar probabilities
7     np = {
8         "John": 0.6,
9         "Mary": 0.4
10    }
11    vp = {
12        "runs": 0.5,
13        "walks": 0.5
14    }
15    # Check validity
16    if words[0] not in np:
17        return 0.0
18    if words[1] not in vp:
19        return 0.0
20    # Calculate probability
21    probability = 1.0 * np[words[0]] * vp[words[1]]
22    return probability
23
24 # Main Program
25 sentence = input("Enter sentence: ")
26 prob = sentence_probability(sentence)
27 print("Sentence Probability = ", round(prob, 2))

```

The output panel on the right shows the execution results for the input "John runs":

```

Enter sentence: John runs
Sentence Probability = 0.3
=== Code Execution Successful ===

```

The bottom status bar indicates the system temperature is 32°C, mostly cloudy, and the time is 11:38 on 06-08-2026.